Phone sensors and hardware APIs

Motion, location, camera and NFC, and what each one costs

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Spring 2027

TODAY

What you should leave with

Know the sensor deck

What each sensor in a phone measures, in which unit, and at what rate.

Read a sensor from Dart

The sensors_plus streams, the sampling period, and where the processing belongs.

Location, done properly

Accuracy modes, the permission flow on both platforms, and the background rules.

Pay for what you use

The energy cost of each sensor, batching, and when to drop to a platform channel.

PART 1 · SENSORS

The hardware already in your pocket

What each sensor measures, its unit, and what fusion adds

The sensors a phone gives you

Accelerometer

Acceleration per axis, gravity included.

Gyroscope

Angular velocity per axis. It drifts when integrated.

Magnetometer

The local magnetic field. Also finds every metal desk.

Barometer

Air pressure. Not on every phone. Relative altitude.

Ambient light

Illuminance at the screen. The OS reads it first.

Proximity

Near or far, over the earpiece. Two states in practice.

These are the sensors, not the features. Step counts, orientation and compass headings are computed from these six, mostly outside your app.

What each one measures, and in what unit

SENSOR	QUANTITY	UNIT	TYPICAL RATE	AT REST
Accelerometer	acceleration per axis	m/s^2	50–200 Hz	one axis near 9.81
Gyroscope	angular velocity	rad/s	50–200 Hz	near 0 on all three
Magnetometer	magnetic flux density	μT	10–100 Hz	25–65 in total
Barometer	air pressure	hPa	1–25 Hz	near 1013 at sea level
Ambient light	illuminance	lx	on change	follows the room
Proximity	distance to an obstacle	cm	on change	far

One hectopascal is roughly eight meters of altitude, so a barometer resolves a staircase but not a street address.

A sampling rate is a request, not a promise

Three delivery models

Continuous sensors push a sample every period. On-change sensors push only when the value moves. One-shot sensors fire once and disarm.

Motion sensors and the barometer are continuous. Light and proximity are on-change.

What you actually get

The rate is a hint. The platform rounds it to what the hardware supports, and another app asking for a faster rate raises yours too.

Samples arrive with jitter. Never assume the gap equals the period you asked for.

Measure the real interval before you trust it. Anything that integrates a signal, such as turning angular velocity into an angle, needs the true time step.

Fusion: what the platform computes for you

Gravity

Which way is down, in m/s^2. A slow low-pass over the accelerometer, corrected by the gyroscope.

Linear acceleration

The accelerometer with gravity subtracted. Near zero when the phone is still, on any face.

Rotation vector

Device orientation as a quaternion, fused from accelerometer, gyroscope and magnetometer.

TWO CHEAPER VARIANTS

Game rotation vector drops the magnetometer: no absolute north, but immune to a metal desk. Geomagnetic rotation vector drops the gyroscope: lower power, slower to settle.

Fusion runs below your app, often on a separate sensor hub. Writing your own filter in Dart is a good exercise and almost always the wrong production answer.

The sensor axes do not rotate with the screen

The device frame
X points right, Y to the top of the screen, Z out of the display, all defined for the natural orientation of the device.

A tablet's natural orientation is landscape, so the same code reports swapped axes there.

Two frames, not one
Sensor values live in the device frame. What you draw on a map lives in the world frame. The rotation vector converts between them.

Turning the phone rotates your widgets. It does not rotate the sensor axes.

Prefer a magnitude over a single axis. The length of the acceleration vector is the same however the user holds the phone.

PART 2 · READING SENSORS

From the sensor hub into your Dart code

sensors_plus, sampling periods, and a step counter you can ship

Four streams, one shape

```dart
import 'package:sensors_plus/sensors_plus.dart';

// Raw accelerometer, gravity included. Near 9.81 at rest.
_raw = accelerometerEventStream().listen(_onRaw);
// Gravity removed by the platform. Near zero when still.
_linear = userAccelerometerEventStream().listen(_onLinear);
// Angular velocity, radians per second, one value per axis.
_spin = gyroscopeEventStream().listen(_onSpin);
// Magnetic field in microtesla. Expect noise indoors.
_field = magnetometerEventStream().listen(_onField);
```

Every event carries `x`, `y` and `z`; the barometer carries one pressure value. Keep each subscription and cancel it in `dispose`, or the sensor stays on.

samplingPeriod: the only knob that matters

```
accelerometerEventStream(
  samplingPeriod: SensorInterval.gameInterval,   // about 50 Hz
).listen(_onSample);

SensorInterval.normalInterval    // about 5 Hz: a value on a screen
SensorInterval.uiInterval        // about 15 Hz: a live gauge
SensorInterval.gameInterval      // about 50 Hz: step detection
SensorInterval.fastestInterval   // whatever the hardware can do
```

Halving the period roughly doubles the cost. Not the cost of the sensor die, which is tiny, but of waking the application processor twice as often to hand a sample to Dart.

Keep the samples off the UI isolate

What every event costs

Each sample crosses from the platform into Dart and wakes the event loop. At 50 Hz that is fifty wake-ups a second competing with your frame budget.

Calling `setState` per sample is the classic mistake.

Where the work goes

Filters, thresholds and counters are a few floating point operations. Keep them here and emit to the widget tree at a human rate.

A window going through an FFT or a model belongs in an isolate (Lecture 2).

Sample fast, render slow. Consume every sample the algorithm needs, then throttle what reaches the widget tree to something the eye can follow.

STEP COUNTER

A step, seen from the accelerometer

What a footfall looks like

The magnitude of the acceleration vector sits near 9.81 while you stand still, then rises and falls once per step.

Walking is roughly 1.5 to 2.5 steps a second. Running is faster and much larger.

Three parts to the algorithm

A slowly moving baseline, so the phone can be held any way. A threshold above it. A debounce, so one footfall cannot count twice.

Nothing here integrates anything. Integration is where drift enters.

Peak counting beats anything clever, at this size. A threshold with hysteresis and a minimum interval is about twenty lines and gets within a few percent on a walk.

STEP COUNTER

Part one: magnitude and a moving baseline

```dart
import 'dart:math';

double _baseline = 9.81;   // slow estimate of "standing still"
bool _above = false;       // hysteresis state
DateTime _last = DateTime.fromMillisecondsSinceEpoch(0);
int steps = 0;

void _onSample(AccelerometerEvent e) {
  final m = sqrt(e.x * e.x + e.y * e.y + e.z * e.z);
  _baseline = 0.9 * _baseline + 0.1 * m;   // a one-pole low-pass
  _detect(m, _baseline);
}
```

The magnitude discards orientation, so pocket, hand and bag behave the same.

Part two: a threshold and a debounce

```
const _minGap = Duration(milliseconds: 250);  // at most 4 steps a second
void _detect(double m, double baseline) {
  final now = DateTime.now();
  if (!_above && m > baseline + 1.2) {           // rising edge
    _above = true;
    if (now.difference(_last) < _minGap) return; // debounce
    steps++;
    _last = now;
  } else if (_above && m < baseline + 0.4) {     // hysteresis band
    _above = false;
  }
}
```

Two thresholds, not one: rearming below 0.4 stops a noisy peak counting twice.

Tuning it, and what will break

The three numbers

The rise threshold sets sensitivity, the fall threshold sets noise immunity, the gap sets the ceiling. Change one at a time over a known hundred steps.

Log the raw magnitude during a walk, then replay the file into the detector.

Known failure modes

A phone on a table during a bus ride counts steps. A swinging bag double counts. A phone held still while the user walks counts almost nothing.

Both platforms ship a hardware step counter on the sensor hub.

Write this once to understand it, then use the platform’s counter. The hardware counter survives your app being killed and never wakes the main processor.

PART 3 · LOCATION

Where the phone thinks it is

Accuracy modes, the permission flow, and the background rules

LOCATION

Where a position actually comes from

Satellites

GNSS: GPS, Galileo, GLONASS, BeiDou. Meters outdoors, nothing indoors, slow from cold.

Wi-Fi and cell

Nearby network identifiers matched against a database. Coarse, fast, and works indoors.

Motion sensors

Dead reckoning between fixes, and why a map pin keeps moving inside a tunnel.

YOU DO NOT PICK THE SOURCE

You state an accuracy and the platform's fused provider picks the cheapest combination that meets it. Asking for less accuracy is how you ask for less power.

Every position carries its own error estimate. `Position.accuracy` is a radius in meters. A fix with a two kilometer radius is not a location.

geolocator accuracy modes and what they cost

ACCURACY MODE	ROUGHLY	POWER	USE IT FOR
reduced	kilometers	lowest	the approximate mode on iOS
lowest	a few km	lowest	a country or city guess
low	about 1 km	low	coarse analytics, a time zone
medium	100–500 m	medium	a ”near you” list
high	under 100 m	high	a map pin, geofencing, the lab
bestForNavigation	best available	highest	turn by turn, on screen only

The mode is a request. With no satellite lock, asking for `high` gets a network fix and a large accuracy radius, not an error.

Declare first, then ask

```
<!-- android/app/src/main/AndroidManifest.xml -->
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>

<!-- ios/Runner/Info.plist -->
<key>NSLocationWhenInUseUsageDescription</key>
<string>Shows the route you walked while the app is open.</string>
```

An undeclared Android permission is denied forever, with no dialog. A missing iOS usage string terminates the app the moment it touches the API.

The sentence in the plist is the product. The user sees it once, inside a dialog you cannot restyle. Say what they gain, not what the app needs.

The permission flow, in code

```
Future<bool> ensureLocation() async {
  var p = await Geolocator.checkPermission();
  if (p == LocationPermission.denied) {
    p = await Geolocator.requestPermission();   // one dialog, in context
  }
  if (p == LocationPermission.deniedForever) {
    await Geolocator.openAppSettings();         // request() is a no-op now
    return false;
  }
  return p == LocationPermission.whileInUse || p == LocationPermission.always;
}
```

Check `isLocationServiceEnabled` first: a granted permission with the radio off returns nothing.

LOCATION

Precise or approximate, while using or always

QUESTION	ANDROID	IOS
How exact	Android 12 adds a Precise or Approximate choice in the dialog	Precise Location can be switched off per app
How long	While using the app, or Only this time, which expires	While Using the App, or Always, prompted later
Getting more	Ask again with a rationale; a second refusal is final	Ask for temporary full accuracy, with its own plist key

Design for approximate first, or the feature is broken for a large share of your users.

Background location: what is allowed at all

Android

A foreground service with the location type and its visible notification keeps updates running. A truly background fix also needs the separate background permission.

From Android 11 that one is granted only from Settings, not from your dialog.

iOS

Always authorization, the location background mode, and an explicit opt-in on the settings object. The status bar tells the user you are tracking.

Significant location change and region monitoring can relaunch a terminated app.

If the user cannot see why you track them, you will lose the permission. Both platforms show periodic reminders naming apps that used location in the background.

A position stream, with a distance filter

```
const settings = LocationSettings(
  accuracy: LocationAccuracy.high,
  distanceFilter: 10,   // meters moved before the next event
);

_positions = Geolocator.getPositionStream(locationSettings: settings)
    .listen((Position p) => _show(p.latitude, p.longitude, p.accuracy));

@override
void dispose() { _positions?.cancel(); super.dispose(); }
```

The distance filter is the cheapest optimization here: a phone on a desk produces no events at all. Use `getCurrentPosition` when one fix is enough, and never leave a stream behind a screen the user has left.

PART 4 · CAMERA AND NFC

Two hardware APIs with very different rules

Controllers, frames, tags, and what iOS will not let you do

The camera controller and its lifecycle

```
late CameraController _c;
Future<void> _start() async {
  final cameras = await availableCameras();   // back, front, external
  _c = CameraController(cameras.first, ResolutionPreset.medium);
  await _c.initialize();                      // opens the hardware
  if (mounted) setState(() {});
}
Widget build(BuildContext ctx) =>
    _c.value.isInitialized ? CameraPreview(_c) : const SizedBox();
```

Call `dispose` on the controller, always. The camera is exclusive: another app taking it, or the phone locking, invalidates the controller.

Two ways to get pixels out

takePicture
Returns an `XFile` on disk at full sensor resolution, with the pipeline's processing applied. One call, one image, no back-pressure.

Use it for anything the user chose to capture: a photo, a document, a receipt.

startImageStream
Hands you a `CameraImage` per preview frame, in the sensor's native format, roughly thirty times a second. No file, no encoding.

Use it when the app decides what is interesting: a barcode, a face, a gauge.

Pick the stream only when a photo will not do. A live stream keeps the sensor, the image pipeline and your analysis running together, which is the most expensive thing in this lecture.

CAMERA

An image stream is a back-pressure problem

```
bool _busy = false;

await _c.startImageStream((CameraImage frame) async {
  if (_busy) return;        // drop frames, never queue them
  _busy = true;
  try {
    await _analyze(frame);  // a detector, a model, your own math
  } finally {
    _busy = false;
  }
});
```

Thirty frames a second is one every 33 milliseconds. If the analysis takes longer, a queue grows without bound. Call `stopImageStream` once the screen is hidden.

NFC: tags, NDEF, and two very different platforms

What a tag is

A passive chip with a few hundred bytes, powered by the reader's own field. It holds one NDEF message: a list of records, each with a type and a payload.

NDEF is the NFC Data Exchange Format. Common records are a URI and a text string.

What each platform allows

Android reads and writes tags in the foreground and can also act as a card. iOS reads and writes NDEF, shows its own scan sheet, and offers you no card emulation.

iOS also needs the tag reading capability and an entitlement.

Design for a tap, not for a session. On both platforms a scan is a short, user-initiated moment. Nothing about NFC gives you a link you can keep open.

One session, one tag, in Dart

```dart
if (!await NfcManager.instance.isAvailable()) return;  // no reader, or off

NfcManager.instance.startSession(onDiscovered: (NfcTag tag) async {
  final ndef = Ndef.from(tag);
  if (ndef == null) {
    await NfcManager.instance.stopSession(errorMessage: 'Not an NDEF tag');
    return;
  }
  _show(ndef.cachedMessage);
  await NfcManager.instance.stopSession();
});
```

Check availability, start a session, inspect the tag, always stop it. Signatures in nfc_manager have moved across major releases, so read the README.

PART 5 · ENERGY

Paying for what you switch on

Cost per sensor, batching, and the platform boundary

What each sensor costs, roughly

WHAT IS ON	ORDER OF DRAW	WHAT THAT MEANS FOR YOUR APP
Barometer, slow rate	well under a milliamp	effectively free, leave it on
Accelerometer at 50 Hz	around a milliamp	cheap, but the wake-ups are not
Gyroscope, continuous	a few milliamps	the priciest motion sensor
Magnetometer plus fusion	a few milliamps	plus continuous work on the processor
GNSS fix, continuous	tens of milliamps	the largest single sensor cost
Camera preview and encode	hundreds of milliamps	budget it in minutes, not hours

The sensor die is rarely the bill. What costs is keeping the processor awake to receive samples, and keeping a radio or an image pipeline powered.

Batching: let the sensor hub hold the samples

The idea

The hub has a small hardware queue. Ask for a maximum report latency and it fills the queue while the processor sleeps, then delivers the batch in one wake-up.

Every sample keeps its own timestamp, so nothing is lost.

When it applies

Only to work that can wait: a fitness log, an activity history, a vibration record. Never to anything you draw on screen right now.

The queue is finite. When it fills, the hub wakes early or drops the oldest samples.

This is the same wake, send, sleep loop as the microcontroller. A sensor hub is a small always-on core with the budget logic from Lecture 3, hidden behind the OS.

ENERGY

When a platform channel is the right tool

Sensors with no plugin

Ambient light and proximity are not in sensors_plus. Both are one native call.

The hardware step counter

It keeps counting while your app is dead. No Dart API reaches it.

Batching and wake control

Report latency and significant motion are native concepts with no Dart surface.

THE SHAPE

A `MethodChannel` for one-off calls, an `EventChannel` for a stream of sensor events. Lecture 12 covers both, and what a channel call costs.

Search pub.dev before you write native code. A channel is right when no maintained plugin exists, or when the plugin hides the knob you need.

Design rules for the lab and the project

Ask late, stop early

Subscribe when the screen appears, cancel when it leaves.

Ask for less

The slowest rate and coarsest accuracy that still works.

Ask in context

Request it when the user taps the feature, never on launch.

Summarize at the edge

Send a step count, not a sample stream.

Degrade, do not dead-end

A denied camera must still leave a working feature.

Measure one number

Sensor rate against battery drain is easy to defend.

Every rule here is checkable in a code review. That is how Lab 4 is graded.

RECAP

Three things to remember

1. A sensor subscription is an energy decision. The rate and the duration cost far more than the code that reads it.
2. Ask for the least you can work with. The coarsest accuracy, the slowest interval, the narrowest permission, requested in context.
3. Fusion, batching and step counting already exist below your app. Reach for the platform before you write the math yourself.

FURTHER READING

pub.dev/packages/sensors_plus · geolocator · camera · Android sensors guide

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel